

audio_outline/final.asm

```
1 ; =====
2 ; main.asm
3 ; Description: Main execution loop and test invocation.
4 ; =====
5 .include "pinball_defs.inc"
6
7 ; Main
8 ; -----
9 ; Description:      System entry point. Initializes stack, calls initialization
10 ;                routines, and runs the EEROMSoundTest.
11 ; Operation:       Sets up the Stack Pointer, calls SysInit, and calls the
12 ;                external EEROMSoundTest. Enters an infinite loop upon return.
13 ; Arguments:       None.
14 ; Return Value:    None.
15 ; Local Variables: None.
16 ; Shared Variables: None.
17 ; Registers Changed: SPH, SPL, R16.
18
19 Main:
20     Initialize Stack Pointer/Vector Table
21     CALL InitFunctions
22     CALL HW4TEST.asm test functions
23
24
25
26
27 ; =====
28 ; vars.inc
29 ; Description: symbol definitions, macros, and constants.
30 ; =====
31
32 ; hardware
33 .equ F_CPU = 8000000           ; CPU Clock Frequency in Hz
34 .equ PRESCALER = 64           ; Timer 1 Prescaler
35
36 ; Actuator Port
37 .equ ACTUATOR_PORT = PORTA     ; Port assigned to drive actuators
38 .equ ACTUATOR_DIR  = DDRA     ; Direction register for actuators
39
40 ; Sound Port Timer 1 OC1A
41 .equ SOUND_PORT = PORTB
42 .equ SOUND_DIR  = DDRB
43 .equ SPEAKER_PIN = 5           ; OC1A is PB5 on ATmega64
44
45 ; SPI / EEROM Pin Port B
46 .equ SPI_PORT = PORTB
47 .equ SPI_DIR  = DDRB
48 .equ SPI_SS   = 0             ; Chip Select for 93C46
49 .equ SPI_SCK  = 1             ; Serial Clock
50 .equ SPI_MOSI = 2             ; Master Out Slave In
51 .equ SPI_MISO = 3             ; Master In Slave Out
52
53 ; 93C46 Opcodes (including Start Bit = 1)
54 .equ EE_READ  = 0b00000110    ; Start (1) + Read Opcode (10)
```

```

55 .equ EE_WRITE = 0b00000101      ; Start (1) + Write Opcode (01)
56 .equ EE_EWEN  = 0b00000100      ; Start (1) + EWEN Opcode (00) (Data requires 11xxxxx)
57 .equ EE_EWDS  = 0b00000100      ; Start (1) + EWDS Opcode (00) (Data requires 00xxxxx)
58
59
60
61
62
63 ; =====
64 ; hardware.asm
65 ; Description: Hardware initialization master routine.
66 ; =====
67
68
69
70 ; InitActuator
71 ; -----
72 ; Description:      Initializes the actuator port as an output and clears it.
73 ; Operation:       Sets ACTUATOR_DIR to all 1s (outputs) and ACTUATOR_PORT to 0.
74 ; Arguments:       None.
75 ; Return Value:    None.
76 ; Registers Changed: R16, ACTUATOR_DIR, ACTUATOR_PORT.
77
78 InitActuator:
79     Set ACTUATOR_DIR to 0xFF
80     Set ACTUATOR_PORT to 0x00
81     RETURN
82
83 ; SetActuator
84 ; -----
85 ; Description:      Turns a specific pinball machine actuator on or off.
86 ; Operation:       Reads the current state of ACTUATOR_PORT. Creates a bitmask
87 ;                  using the actuator number a. If state s is TRUE, it
88 ;                  ORs the mask. If FALSE, it ANDs the inverted mask.
89 ; Arguments:       R16 - Actuator number a (0-7).
90 ;                  R17 - Actuator state s (0 = FALSE, non-zero = TRUE).
91 ; Return Value:    None.
92 ; Local Variables: Bitmask.
93 ; Shared Variables: ACTUATOR_PORT (Read/Write).
94 ; Registers Changed: R16, R17, R18.
95
96 SetActuator:
97     Create Bitmask: R18 = (1 << R16 (a))
98     Read current port state into a temp register
99
100     IF (R17 (s) != 0) THEN
101         Set the target bit: Temp = Temp OR R18
102     ELSE
103         Clear the target bit: Temp = Temp AND (NOT R18)
104     ENDIF
105
106     Write Temp back to ACTUATOR_PORT
107     RETURN
108
109
110 ; InitEEROM

```

```

111 ; -----
112 ; Description:      Initializes hardware SPI for EEROM communication.
113 ; Operation:       Sets SCK, MOSI, and SS pins as outputs, MISO as input.
114 ;                 Enables SPI in Master Mode.
115 ; Arguments:       None.
116 ; Return Value:    None.
117 ; Registers Changed: R16, SPI_DIR, SPI_PORT, SPCR.
118
119 InitEEROM:
120     Set SPI_DIR bits for SCK, MOSI, and SS to 1 (Output)
121     Set SPI_DIR bit for MISO to 0 (Input)
122     Set SS Pin high (Inactive)
123     Enable SPI, Master mode, Clock Rate fck/16 (Write to SPCR)
124     RETURN
125
126 ; InitSound
127 ; -----
128 ; Description:      Configures Timer 1 for sound generation.
129 ; Operation:       Sets SPEAKER_PIN to output. Configures Timer 1 in CTC mode
130 ;                 (Clear Timer on Compare match OCR1A), toggling OC1A.
131 ; Arguments:       None.
132 ; Return Value:    None.
133 ; Registers Changed: R16, TCCR1A, TCCR1B, SOUND_DIR.
134
135 InitSound:
136     Set SPEAKER_PIN bit in SOUND_DIR to 1 (Output)
137     Set TCCR1A = (1 << COM1A0) ; Toggle OC1A on compare match
138     Set TCCR1B = (1 << WGM12) | PRESCALER ; CTC mode, prescaler set
139     Set OCR1A to 0 ; Default to 0
140     RETURN
141
142
143
144
145 ; =====
146 ; eerom.asm
147 ; Description: Serial EEPROM (93C46) access routines via Hardware SPI.
148 ; =====
149
150 ; SPITransfer
151 ; -----
152 ; Description:      Sends and receives one byte over hardware SPI.
153 ; Operation:       Loads data into SPDR, waits for the SPIF flag, reads SPDR.
154 ; Arguments:       R16 - Byte to send.
155 ; Return Value:    R16 - Received byte.
156 ; Registers Changed: R16, SPSR, SPDR.
157
158 SPITransfer:
159     Write R16 to SPDR
160 WaitLoop:
161     Read SPSR
162     IF (SPIF bit is 0) GOTO WaitLoop
163     Read SPDR into R16
164     RETURN
165
166 ; ReadEEROM

```

```

167 ; -----
168 ; Description:      Reads n bytes from EEROM at address a into buffer p
169 ; Operation:       Loops n times. For each iteration, asserts CS, constructs
170 ;                  the READ opcode + address bitstream, shifts it via SPI,
171 ;                  reads the data byte, and deasserts CS.
172 ; Arguments:       R16 - Number of bytes n.
173 ;                  R17 - Starting EEROM address a.
174 ;                  Y (R29|R28) - Pointer p to RAM buffer.
175 ; Return Value:    None.
176 ; Local Variables: ByteCounter, AddressCounter, HighCmd, LowCmd.
177 ; Shared Variables: RAM Buffer.
178 ; Registers Changed: R16, R17, R18, R19, Y.
179
180 ReadEEROM:
181     AddressCounter = R17
182     ByteCounter = R16
183
184 ReadLoop:
185     IF (ByteCounter == 0) GOTO EndRead
186
187     Assert SS Pin (Set Low/High depending on active state, usually High for 93C46)
188
189     ; Construct 10-bit Read Command: Start(1) + Op(10) + Addr(7-bit)
190     ; Sent via two 8-bit SPI transfers.
191     HighCmd = EE_READ
192     LowCmd = (AddressCounter << 1) ; Shift 7-bit address to align
193
194     Call SPITransfer(HighCmd)
195     Call SPITransfer(LowCmd)
196
197     Call SPITransfer(0x00) ; Send byte to clock in the data
198     Store Received Byte in RAM at Y
199     Increment Y
200
201     Deassert SS Pin (Drive Low)
202
203     Increment AddressCounter
204     Decrement ByteCounter
205     GOTO ReadLoop
206
207 EndRead:
208     RETURN
209
210 ; WriteEEROM
211 ; -----
212 ; Description:      Writes n bytes to EEROM at address a from buffer p.
213 ; Operation:       Issues an EWEN command. Loops n times sending the WRITE
214 ;                  opcode + address, followed by data. Waits for the write
215 ;                  cycle to complete (polls MISO). Finally issues an EWDS command.
216 ; Arguments:       R16 - Number of bytes n.
217 ;                  R17 - Starting EEROM address a.
218 ;                  Y (R29|R28) - Pointer p to RAM buffer.
219 ; Return Value:    None.
220 ; Local Variables: ByteCounter, AddressCounter, HighCmd, LowCmd, DataByte.
221 ; Shared Variables: RAM Buffer.
222 ; Registers Changed: R16-R20, Y.

```

```

223
224 WriteEEROM:
225     ; Enable Erase/Write
226     Assert SS Pin
227     Call SPITransfer(EWEN)
228     Call SPITransfer(0b11000000) ; 93C46 requires 11x for EWEN data phase
229     Deassert SS Pin
230
231     AddressCounter = R17
232     ByteCounter = R16
233
234 WriteLoop:
235     IF (ByteCounter == 0) GOTO EndWrite
236
237     ;Write Command and Address
238     Assert SS Pin
239     HighCmd = EE_WRITE
240     LowCmd  = (AddressCounter << 1)
241
242     Call SPITransfer(HighCmd)
243     Call SPITransfer(LowCmd)
244
245     ;Write Data
246     Load DataByte from RAM at Y
247     Call SPITransfer(DataByte)
248     Deassert SS Pin
249
250     ;Poll for completion
251     Assert SS Pin
252 PollReady:
253     Read MISO Pin
254     IF (MISO == 0) GOTO PollReady    ; 0 = Busy 1 = Ready
255     Deassert SS Pin
256
257     Increment Y
258     Increment AddressCounter
259     Decrement ByteCounter
260     GOTO WriteLoop
261
262 EndWrite:
263     ;Disable Erase/Write
264     Assert SS Pin
265     Call SPITransfer(EWDS)
266     Call SPITransfer(0b00000000) ; 93C46 needs 00x for EWDS data phase
267     Deassert SS Pin
268
269     RETURN
270
271
272
273
274
275
276 ; =====
277 ; sound.asm
278 ; Description: Note generation and speaker control routines via Timer 1.

```

```

279 ; =====
280
281
282 ; PlayNote
283 ; -----
284 ; Description:      Plays a note of frequency 'f' Hz on the speaker.
285 ; Operation:        If 'f' is 0, turns off the timer output. If > 0, calculates
286 ;                   the required OCR1A value using (F_CPU / (2 * PRESCALER * f)) - 1
287 ;                   and updates the OCR1AH and OCR1AL registers.
288 ; Arguments:        R17 | R16 - Frequency 'f' in Hz (R17 is High Byte).
289 ; Return Value:      None.
290 ; Local Variables:   OCR_Value.
291 ; Shared Variables:  TCCR1A, OCR1A.
292 ; Registers Changed: R16-R21 (depending on division logic).
293
294 PlayNote:
295     IF (f (R17|R16) == 0) THEN
296         Disconnect Timer 1 from OC1A pin (Clear COM1A0 in TCCR1A)
297         Clear SPEAKER_PIN output to 0
298         RETURN
299     ELSE
300         Reconnect Timer 1 to OC1A pin (Set COM1A0 in TCCR1A)
301     ENDIF
302
303     ; Calculate Timer Compare Value
304     ; OCR1A = (F_CPU / (2 * PRESCALER * f)) - 1
305     Numerator = F_CPU / (2 * PRESCALER) ; constants
306     Denominator = f (R17|R16)
307
308     Call Divide32by16(Numerator, Denominator) ; result in a register pair
309     Subtract 1 from the Result
310
311     Write Result High to OCR1AH
312     Write Result Low to OCR1AL
313
314     RETURN

```